JOHANNES KEPLER
UNIVERSITY LINZ

Design for Test



Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

I I C Q C
INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- relate **yield**, **fault coverage**, and **defect level**
- model defects as **faults**: stuck-at, transition, bridging
- find a **test** for a fault, by hand and with a **SAT solver**
- make flip-flops testable with **scan chains**
- generate vectors with **Fault** and apply them through a chain
- apply the **design rules** for testability
- name **at-speed test**, **BIST**, and **boundary scan**



Chapter 9

Lecture notes: Chapter 9 of iic-jku.github.io/vlsi-design

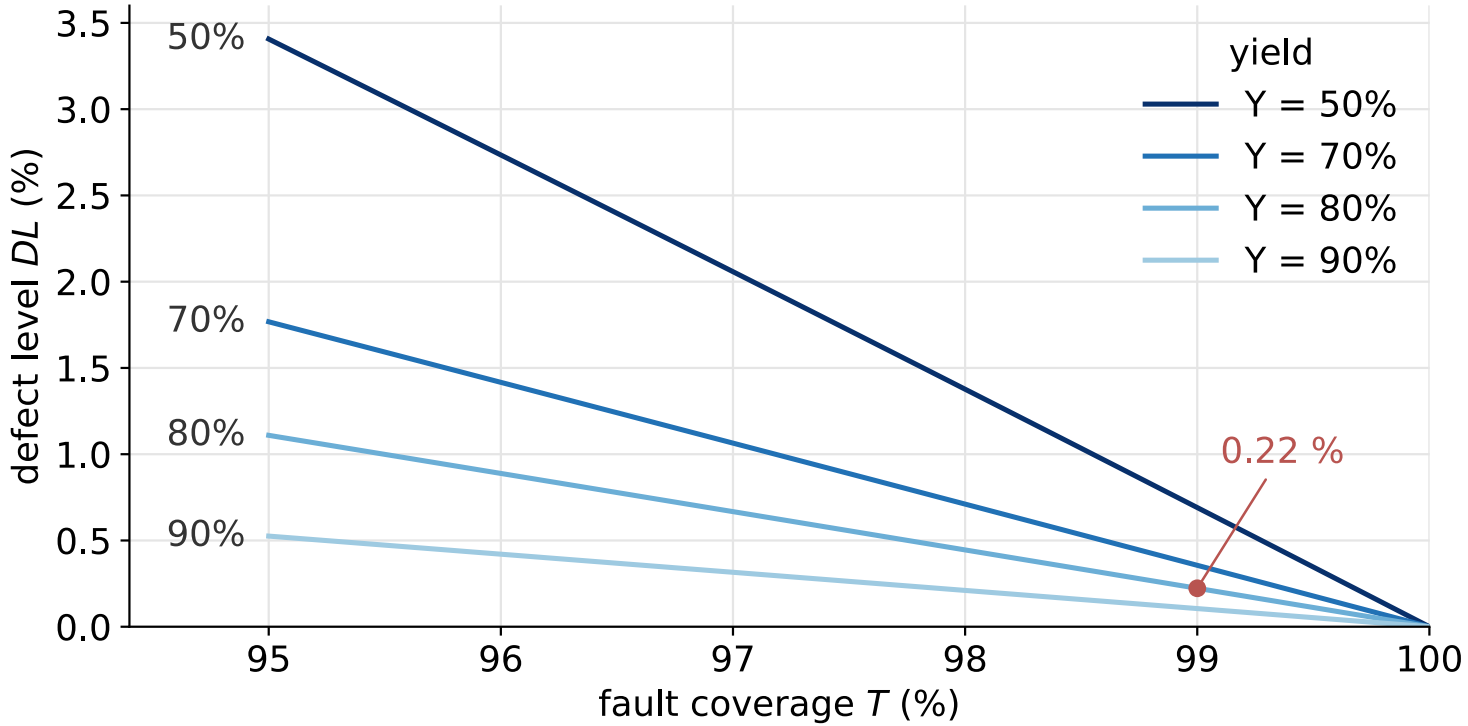
Why Test?



Every Die Is Tested

- Particles, lithography, process variations: **shorts**, **opens**, dead transistors
- **Yield** Y: the share of good dies, never 100 %
- **Verification**: is the design right? **Test**: does this die equal the design?
- **Wafer test** with a probe card, **final test** after packaging, **burn-in** for cars
- **ATE** (automatic test equipment) applies a vector per cycle: seconds per die
- Rule of ten: each later step (package, board, system) costs about 10× more

Defect Level



- **DL = 1 – Y^{1-T}** (Williams and Brown): Y = 80 %, T = 99 %: **0.22 %**, one part in 449
- 100 ppm at 80 % yield needs **T = 99.955 %**

Fault Models



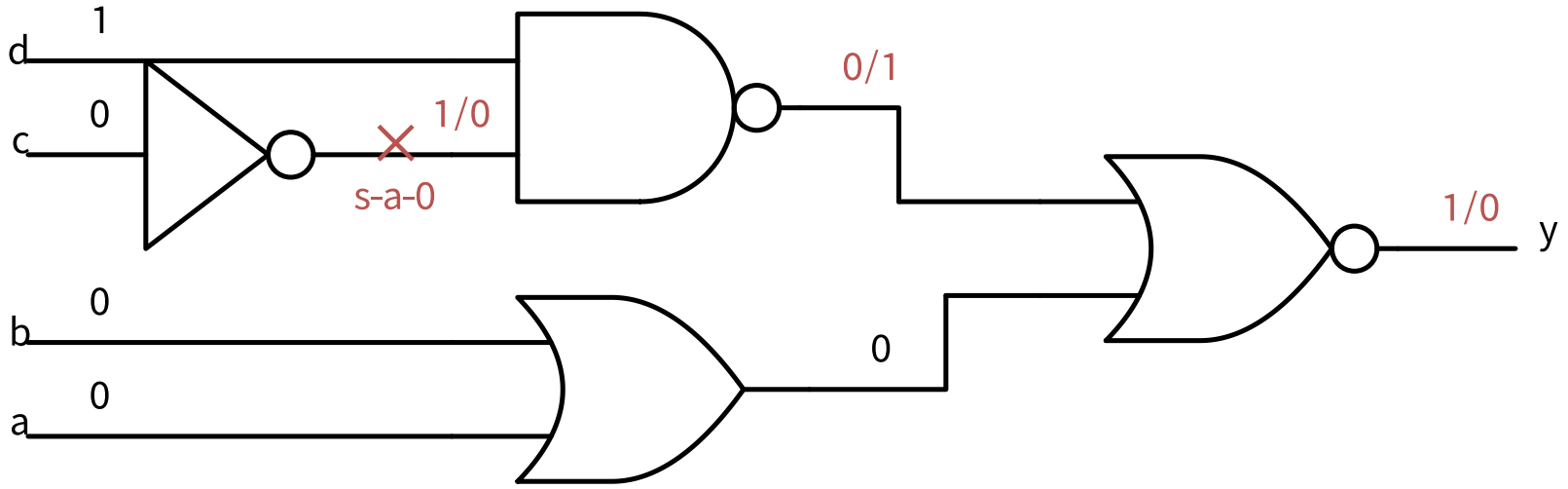
From Defects to Faults

- **Single stuck-at**: one net or pin fixed at 0 or 1, the cells keep their functions
  - n sites: 2n faults, Fault counts every cell pin and port bit
- **Equivalent** faults have the same tests: fault collapsing
- **Untestable** faults in redundant logic: **test coverage** leaves them out
- **Transition** faults: slow-to-rise, slow-to-fall, two patterns at speed
- **Bridging** (two nets shorted), **stuck-open** (two patterns), **IDDQ** (current)

Test Generation



Excite, Propagate, Justify



- **Excite**: drive the fault site to the opposite value, here c = 0
- **Propagate**: other gate inputs non-controlling, d = 1, a = b = 0
- **Justify**: reach all these values from the inputs, or backtrack

Random Vectors and a SAT Solver

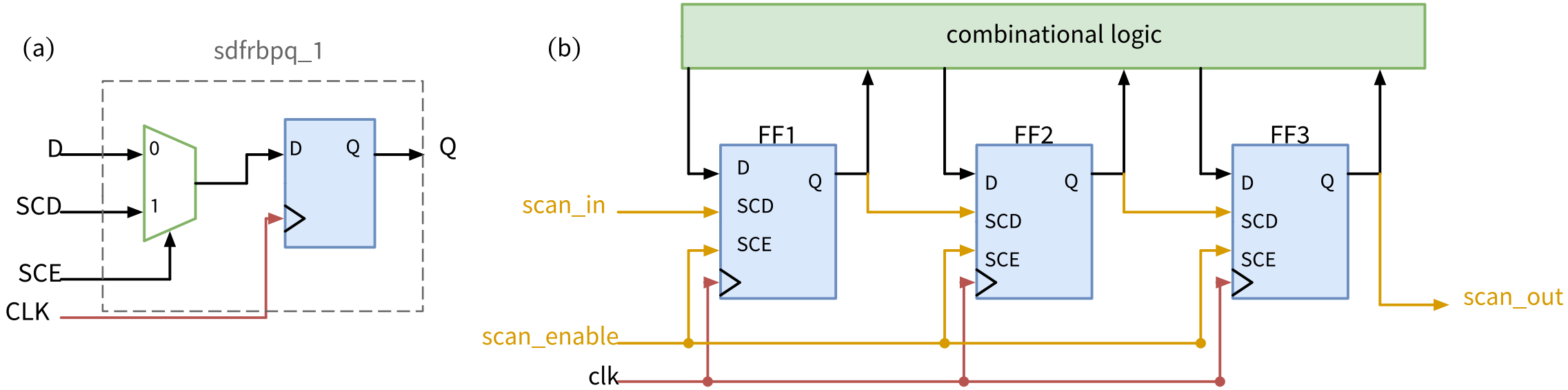
- All vectors? 32-bit barrel shifter: $2^{39} \approx 5.5 \cdot 10^{11}$, **1.5 h per die** at 100 MHz
- **Random vectors** plus **fault simulation** find most faults fast
  - 16-bit comparator with a constant: 1000 vectors detect its output stuck-at-0 with 1.5 %
- **SAT**: a miter of good and faulty netlist, any solution is a test
  - no solution: the fault is proven untestable, `atpg/sat_test.tcl`
- **Compaction** drops vectors whose faults others detect too
- **Fault** 0.9.4: random vectors, Icarus fault simulation, compaction

Exercise (Section 9.4): find tests for four faults by hand, with the SAT solver, and with Fault.

Scan Design

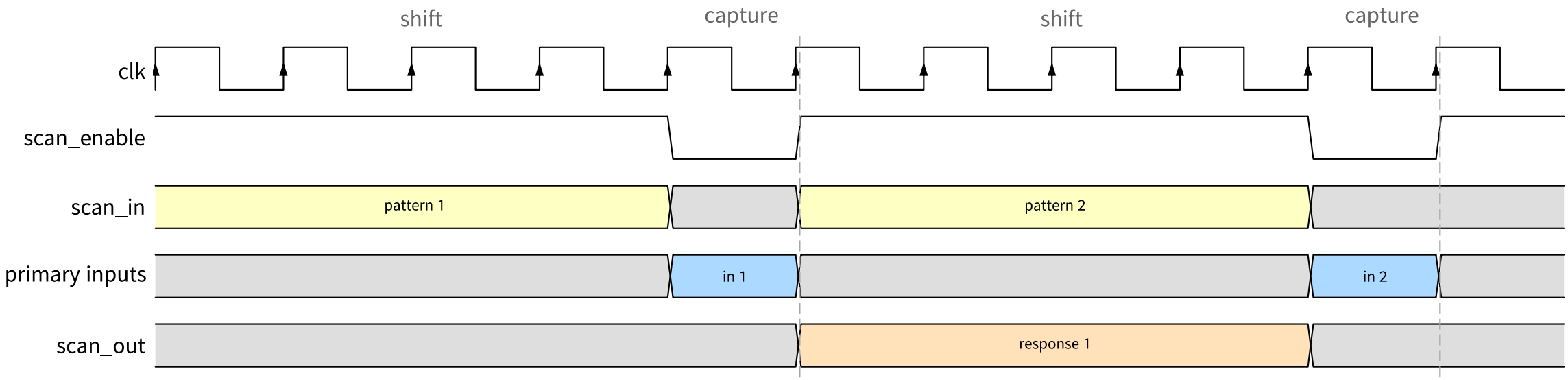


Scan Flip-Flops and Chains



- Every flip-flop becomes **controllable** and **observable**: the test turns combinational
- SG13CMOS5L: `sdfrbpq_1` with scan input `SCD` and enable `SCE`, Liberty `test_cell` groups
- Costs speed in `bs_reg`: setup 0.204 instead of 0.125 ns, period 1.30 to 1.43 ns

Shift, Capture, Shift



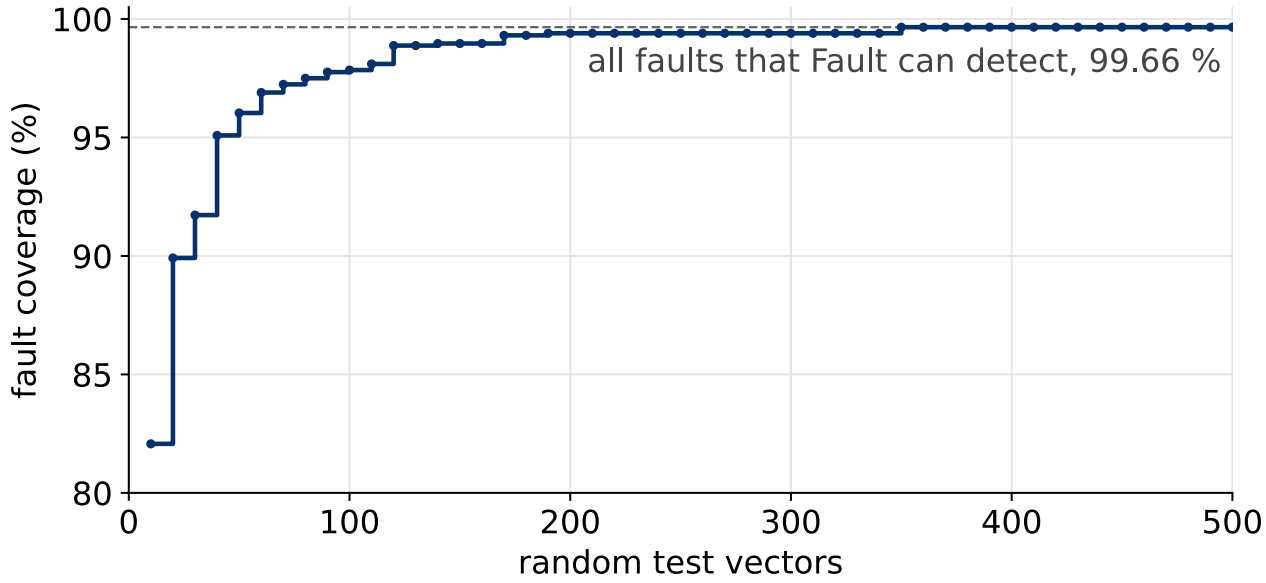
- N patterns, chains of L flip-flops: **(N + 1) L + N** clock cycles
- Several chains in parallel, **test compression** for many short chains
- **Chain test** first: a known sequence through the chain, without capture

Scan in the Open-Source Flow

```
# OpenROAD, scan/dft.tcl: replace the flip-flops, stitch the chains
set_dft_config -max_length $max_length -clock_mixing no_mix
scan_replace
execute_dft_plan
# Fault: flip-flops become inputs and outputs, then random ATPG
fault cut bs_reg.nl.v --sclConfig scl.yml --clock clk_i ...
fault atpg bs_reg.cut.v -c $CELLS/sg13cmos5l_stdcell.v ...
```

- Full flow: `scan_replace` **before** placement, `execute_dft_plan` **after** it
- LibreLane has **no** scan insertion step
- Fault's vectors through the chain: `scan_patterns.py`, testbench `tb/tb_scan.sv`
- Afterwards: **equivalence check** with the scan inputs at 0, `scan/scan_eq.tcl`

Coverage of Random Vectors



- 8-bit `bs_reg`: **1160 faults**, 4 proven untestable by SAT, **32 vectors** after compaction
- One fault per cell pin instead of per net: the 32 vectors reach only **95.4 %**

Exercise (Section 9.5): insert a scan chain and apply Fault's vectors through it.

Design Rules for Testability



Design Rules for Testability

- **Clocks from pins**: dividers and PLLs need a test clock multiplexer
- **Clock gates with a test enable**: `slgcp_1`, input `SCE` to the scan enable
  - clock gating is fine for scan, but Yosys and OpenROAD leave `SCE` open
- **Resets from pins**: no flip-flop may be reset while the chain shifts
- **No latches**, **no combinational loops**, **no pulse generators**
- **No unknowns** at flip-flops: memories, analog, tri-states get a bypass

Resets, Clock Domains, and Hold

- A **generated reset** (power-on reset, synchronizer, flip-flop) needs care in test mode
- **One chain per clock**: the asynchronous FIFO gets chains of 43 and 11
  - crossing chains need a **lockup latch**, crossing paths are captured one domain at a time
- The chain is a timing path: `Q` straight to `SCD`, the **first to fail hold**
  - scanned `bs_reg`: the 20 smallest hold slacks are scan paths, from 0.292 ns

Exercise (Section 9.6): make the barrel shifter behind a reset synchronizer testable.

Beyond Scan



Beyond Scan

- **At-speed test**: launch on capture (LOC) or launch on shift (LOS), PLL clock pulses
- **BIST**: pseudo-random patterns from an LFSR, a signature in a MISR, also in the field
- **Memory BIST**: march patterns for the SRAM macros
- **Boundary scan** (JTAG, IEEE 1149.1): TAP with `TCK`, `TMS`, `TDI`, `TDO`
- **Parametric** tests, speed **binning**, **diagnosis** of failing dies

Summary

- Defect level $\mathbf{1 - Y^{1-T}}$: coverage must be far above 99 %
- **Stuck-at** faults, untestable ones sit in redundant logic
- Excite, propagate, justify: **random vectors**, then **SAT**
- **Scan**: shift, capture, shift, (N + 1) L + N cycles
- Open source: **OpenROAD** inserts the chains, **Fault** generates vectors
- Rules: clocks and resets from pins, clock gates with **test enable**, hold on scan paths
- At-speed test, BIST, and boundary scan complete the test

Exercises

1. Find the Test (9.4)
2. Insert a Scan Chain (9.5)
3. Make It Testable (9.6)

All code in `content/dft/code` of the course repository, every number reproduced by `scripts/check_dft.sh`



Chapter 9



JOHANNES KEPLER
UNIVERSITY LINZ